

Soma Technical Assessment Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: Extend a basic Next.js todo app with due dates, Pexels image previews, and a full task dependency system (circular dependency prevention, critical path, earliest start dates, dependency graph visualization).

Architecture: Server-side graph computation via a dedicated algorithm library. Prisma schema extended with optional fields and an explicit dependency join table. Frontend split into focused components with React Flow for graph visualization.

Tech Stack: Next.js 16 (App Router), React 19, Prisma 7 + SQLite, Tailwind CSS v4, React Flow (@xyflow/react), dagre (graph layout)

File Structure

New files:

- `src/lib/pexels.ts` — Pexels API client (server-only)
- `src/lib/graph.ts` — Graph algorithms (cycle detection, topological sort, critical path, earliest start dates)
- `src/components/ToDoCard.tsx` — Individual todo card with image, due date, dependency management
- `src/components/DependencyGraph.tsx` — React Flow graph visualization with dagre layout

- `src/app/api/todos/[id]/dependencies/route.ts` — Add/remove dependency edges
- `src/app/api/todos/graph/route.ts` — Computed graph data (critical path, earliest starts)

Modified files:

- `prisma/schema.prisma` — Add `dueDate`, `imageUrl`, `TodoDependency` model
- `next.config.ts` — Add Pexels image domain to `remotePatterns`
- `src/app/api/todos/route.ts` — Accept `dueDate` in POST, include dependencies in GET
- `src/app/api/todos/[id]/route.ts` — Cascade-delete dependency edges on todo delete
- `src/app/page.tsx` — Orchestrator with view toggle, data fetching, component composition

New dependencies:

- `@xyflow/react` — React Flow v12 for interactive graph visualization
- `dagre` + `@types/dagre` — Automatic directed graph layout

Task 1: Database Schema Migration

Files:

- Modify: `prisma/schema.prisma`
- ☐ **Step 1: Update the Prisma schema**

Replace the entire contents of `prisma/schema.prisma`:

```
generator client {  
  provider = "prisma-client"
```

```

    output    = "../src/generated/prisma"
  }

  datasource db {
    provider = "sqlite"
  }

  model Todo {
    id          Int          @id @default(autoincrement())
    title       String
    createdAt   DateTime @default(now())
    dueDate     DateTime?
    imageUrl    String?

    dependsOn   TodoDependency[] @relation("dependent")
    dependedOnBy TodoDependency[] @relation("dependency")
  }

  model TodoDependency {
    dependentId Int
    dependencyId Int

    dependent Todo @relation("dependent", fields: [dependentId])
    dependency Todo @relation("dependency", fields: [dependencyId])

    @@id([dependentId, dependencyId])
  }

```

- ☐ **Step 2: Run the migration**

```
npx prisma migrate dev --name add-duedate-image-dependency
```

Expected: Migration created, client regenerated, no errors.

- ☐ **Step 3: Verify the generated client has the new types**

```

grep -r "dueDate" src/generated/prisma/ | head -5
grep -r "TodoDependency" src/generated/prisma/ | head -5

```

Expected: Both fields and model appear in generated output.

- ☐ **Step 4: Commit**

```
git add prisma/schema.prisma prisma/migrations/ src/g
git commit -m "feat: add schema for due dates, images,
```

Task 2: Due Dates

Files:

- Modify: `src/app/api/todos/route.ts`
- Modify: `src/app/page.tsx`
- ☐ **Step 1: Update POST route to accept dueDate**

In `src/app/api/todos/route.ts`, replace the POST handler:

```
export async function POST(request: Request) {
  try {
    const { title, dueDate } = await request.json();
    if (!title || title.trim() === "") {
      return NextResponse.json({ error: "Title is req"
    }
    const todo = await prisma.todo.create({
      data: {
        title,
        dueDate: dueDate ? new Date(dueDate) : null,
      },
    });
    return NextResponse.json(todo, { status: 201 });
  } catch {
    return NextResponse.json({ error: "Error creating
  }
}
```

- ☐ **Step 2: Verify POST with curl**

```
curl -s -X POST http://localhost:3000/api/todos \
  -H 'Content-Type: application/json' \
  -d '{"title":"Test due date","dueDate":"2026-04-01"'
```

Expected: Response includes `"dueDate": "2026-04-01T00:00:00.000Z"`.

- ☐ **Step 3: Wire the frontend date picker to state**

In `src/app/page.tsx`, add `dueDate` state and wire the existing `<input type="date">`:

```
const [dueDate, setDueDate] = useState("");
```

Replace the bare `<input type="date" />` with:

```
<input
  type="date"
  value={dueDate}
  onChange={e => setDueDate(e.target.value)}
  className="border-y border-gray-200 p-3 text-gray-700"
/>
```

- ☐ **Step 4: Send dueDate in the POST request**

In `handleAddTodo`, update the `body`:

```
body: JSON.stringify({ title: newTodo, dueDate: dueDate })
```

And reset `dueDate` after creation:

```
setNewTodo("");  
setDueDate("");
```

- ☐ **Step 5: Display due dates on todo cards with overdue styling**

Replace the `<span>` inside each `<li>` with:

```
<div>  
  <span className="text-gray-800">{todo.title}</span>  
  {todo.dueDate && (  
    <p  
      className={`text-sm ${  
        new Date(todo.dueDate) < new Date()  
          ? "font-semibold text-red-500"  
          : "text-gray-500"  
      }`}  
    >  
      Due: {new Date(todo.dueDate).toLocaleDateString(  
        {new Date(todo.dueDate) < new Date() && " (overdue)"  
      }  
    </p>  
  )}  
</div>
```

- ☐ **Step 6: Verify in browser**

Start dev server (`npm run dev`), create a todo with a past date and one with a future date. Confirm:

- Past date shows in red with "(overdue)"
- Future date shows in gray
- No date shows nothing
- ☐ **Step 7: Commit**

```
git add src/app/api/todos/route.ts src/app/page.tsx
git commit -m "feat: implement due dates with overdue"
```

Task 3: Pexels Image Integration

Files:

- Create: `src/lib/pexels.ts`
- Modify: `next.config.ts`
- Modify: `src/app/api/todos/route.ts`
- ☐ **Step 1: Create the Pexels API client**

Create `src/lib/pexels.ts`:

```
const PEXELS_API_KEY = process.env.PEXELS_API_KEY;

interface PexelsSearchResponse {
  photos: {
    src: {
      landscape: string;
    };
    alt: string;
  }[];
}

export async function searchPhoto(query: string): Promise<PexelsSearchResponse> {
  if (!PEXELS_API_KEY) {
    console.warn("PEXELS_API_KEY not set, skipping image search");
    return null;
  }

  try {
    const res = await fetch(
      `https://api.pexels.com/v1/search?query=${encodeURIComponent(query)}`
    );
    const json = await res.json();
    return json as PexelsSearchResponse;
  } catch {
    return null;
  }
}
```

```

        headers: { Authorization: PEXELS_API_KEY },
      },
    );

    if (!res.ok) return null;

    const data: PexelsSearchResponse = await res.json;
    return data.photos[0]?.src?.landscape ?? null;
  } catch {
    return null;
  }
}

```

- ☐ **Step 2: Add Pexels image domain to Next.js config**

Replace `next.config.ts`:

```

import type { NextConfig } from "next";

export default {
  typedRoutes: true,
  experimental: {
    typedEnv: true,
  },
  images: {
    remotePatterns: [
      {
        protocol: "https",
        hostname: "images.pexels.com",
      },
    ],
  },
} satisfies NextConfig;

```

- ☐ **Step 3: Integrate image fetch into POST route**

In `src/app/api/todos/route.ts`, add the import and update the POST handler:


```
import { searchPhoto } from "@lib/pexels";
```

Update the `prisma.todo.create` call to fetch and store the image:

```
export async function POST(request: Request) {
  try {
    const { title, dueDate } = await request.json();
    if (!title || title.trim() === "") {
      return NextResponse.json({ error: "Title is required" });
    }

    const imageUrl = await searchPhoto(title);

    const todo = await prisma.todo.create({
      data: {
        title,
        dueDate: dueDate ? new Date(dueDate) : null,
        imageUrl,
      },
    });
    return NextResponse.json(todo, { status: 201 });
  } catch {
    return NextResponse.json({ error: "Error creating todo" });
  }
}
```

- ❑ **Step 4: Verify image fetch with curl**

```
curl -s -X POST http://localhost:3000/api/todos \
  -H 'Content-Type: application/json' \
  -d '{"title":"Buy groceries"}' | jq .imageUrl
```

Expected: A Pexels URL like

`"https://images.pexels.com/photos/..."` or `null` if API key is missing.

- ☐ **Step 5: Commit**

```
git add src/lib/pexels.ts next.config.ts src/app/api/t
git commit -m "feat: add Pexels image integration for
```

Note: The frontend image display will be added in Task 5 when extracting the TodoCard component, since image loading state requires per-card state.

Task 4: Task Dependencies — Backend

Files:

- Create: `src/lib/graph.ts`
- Create: `src/app/api/todos/[id]/dependencies/route.ts`
- Create: `src/app/api/todos/graph/route.ts`
- Modify: `src/app/api/todos/route.ts` (GET — include dependencies)
- Modify: `src/app/api/todos/[id]/route.ts` (DELETE — cascade dependencies)
- ☐ **Step 1: Create the graph algorithms library**

Create `src/lib/graph.ts`:

```
interface TodoForGraph {
  id: number;
  dueDate: Date | string | null;
  dependsOn: { dependencyId: number }[];
}

/**
 * Check if adding "dependentId depends on dependency"
```

```

    * Walks from dependencyId following its dependsOn edges
    */
export function wouldCreateCycle(
  todos: TodoForGraph[],
  dependentId: number,
  dependencyId: number,
): boolean {
  const adjList = new Map<number, number[]>();
  for (const todo of todos) {
    adjList.set(
      todo.id,
      todo.dependsOn.map((d) => d.dependencyId),
    );
  }

  const visited = new Set<number>();
  const stack = [dependencyId];

  while (stack.length > 0) {
    const current = stack.pop()!;
    if (current === dependentId) return true;
    if (visited.has(current)) continue;
    visited.add(current);
    for (const dep of adjList.get(current) ?? []) {
      stack.push(dep);
    }
  }

  return false;
}

/**
 * Kahn's algorithm. Returns todo IDs in topological order
 */
export function topologicalSort(todos: TodoForGraph[]) {
  const inDegree = new Map<number, number>();
  const forward = new Map<number, number[]>(); // dependencyId to list of dependents

  for (const todo of todos) {
    inDegree.set(todo.id, todo.dependsOn.length);
    if (!forward.has(todo.id)) forward.set(todo.id, []);
    for (const dep of todo.dependsOn) {
      const list = forward.get(dep.dependencyId) ?? [];
      list.push(todo.id);
      forward.set(dep.dependencyId, list);
    }
  }
}

```

```

    }
  }

  const queue: number[] = [];
  for (const [id, deg] of inDegree) {
    if (deg === 0) queue.push(id);
  }

  const sorted: number[] = [];
  while (queue.length > 0) {
    const current = queue.shift()!;
    sorted.push(current);
    for (const next of forward.get(current) ?? []) {
      const newDeg = inDegree.get(next)! - 1;
      inDegree.set(next, newDeg);
      if (newDeg === 0) queue.push(next);
    }
  }

  // If sorted is shorter than input, there's a cycle
  if (sorted.length !== todos.length) {
    console.error("Cycle detected in dependency graph");
    return sorted; // Return partial result rather than throw
  }

  return sorted;
}

/**
 * Find the critical path (longest path in the DAG by
 * Returns an ordered array of todo IDs from root to leaf
 */
export function findCriticalPath(todos: TodoForGraph[]): number[] {
  if (todos.length === 0) return [];

  const sorted = topologicalSort(todos);
  const todoMap = new Map(todos.map((t) => [t.id, t]));

  // dist[id] = length of longest path ending at id
  const dist = new Map<number, number>();
  const prev = new Map<number, number | null>();

  for (const id of sorted) {
    dist.set(id, 0);
    prev.set(id, null);
  }
}

```

```

    }

    for (const id of sorted) {
        const todo = todoMap.get(id)!;
        for (const dep of todo.dependsOn) {
            const candidate = dist.get(dep.dependencyId)! +
            if (candidate > dist.get(id)!) {
                dist.set(id, candidate);
                prev.set(id, dep.dependencyId);
            }
        }
    }
}

// Find the endpoint with maximum distance
let maxDist = 0;
let endNode = sorted[0];
for (const [id, d] of dist) {
    if (d > maxDist) {
        maxDist = d;
        endNode = id;
    }
}

// Trace back from endpoint to root
const path: number[] = [];
let current: number | null | undefined = endNode;
while (current != null) {
    path.unshift(current);
    current = prev.get(current);
}

return path;
}

/**
 * Compute the earliest possible start date for each task
 * - No dependencies: today
 * - Has dependencies: max(dependency due dates or the task's due date)
 */
export function computeEarliestStartDates(
    todos: TodoForGraph[],
): Record<number, string> {
    const sorted = topologicalSort(todos);
    const todoMap = new Map(todos.map((t) => [t.id, t]));
    const today = new Date();

```

```

today.setHours(0, 0, 0, 0);

const earliest = new Map<number, Date>();

for (const id of sorted) {
  const todo = todoMap.get(id)!;
  if (todo.dependsOn.length === 0) {
    earliest.set(id, new Date(today));
  } else {
    let maxDate = new Date(today);
    for (const dep of todo.dependsOn) {
      const depTodo = todoMap.get(dep.dependencyId);
      const depDate = depTodo?.dueDate
        ? new Date(depTodo.dueDate)
        : (earliest.get(dep.dependencyId) ?? today);
      if (depDate > maxDate) {
        maxDate = new Date(depDate);
      }
    }
    earliest.set(id, maxDate);
  }
}

const result: Record<number, string> = {};
for (const [id, date] of earliest) {
  result[id] = date.toISOString();
}
return result;
}

```

- ❑ **Step 2: Create the dependencies API route**

Create `src/app/api/todos/[id]/dependencies/route.ts`:

Note: Next.js 15+ requires `params` to be awaited. All `[id]` route handlers use the async pattern.

```

import { NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";
import { wouldCreateCycle } from "@/lib/graph";

export async function POST(

```

```

    request: Request,
    { params }: { params: Promise<{ id: string }> },
  ) {
    const { id } = await params;
    const dependentId = parseInt(id);
    const { dependencyId } = await request.json();

    if (isNaN(dependentId) || isNaN(dependencyId)) {
      return NextResponse.json({ error: "Invalid IDs" }, {
        status: 400,
      });
    }

    if (dependentId === dependencyId) {
      return NextResponse.json(
        { error: "A task cannot depend on itself" },
        { status: 400 },
      );
    }

    // Check for circular dependency
    const allTodos = await prisma.todo.findMany({
      include: { dependsOn: true },
    });

    if (wouldCreateCycle(allTodos, dependentId, dependencyId)) {
      return NextResponse.json(
        { error: "This would create a circular dependency" },
        { status: 400 },
      );
    }

    try {
      const dep = await prisma.todoDependency.create({
        data: { dependentId, dependencyId },
      });
      return NextResponse.json(dep, { status: 201 });
    } catch {
      return NextResponse.json(
        { error: "Dependency already exists or invalid" },
        { status: 400 },
      );
    }
  }
}

export async function DELETE(
  request: Request,

```

```

    { params }: { params: Promise<{ id: string }> },
  ) {
    const { id } = await params;
    const dependentId = parseInt(id);
    const { dependencyId } = await request.json();

    try {
      await prisma.todoDependency.delete({
        where: {
          dependentId_dependencyId: { dependentId, dependencyId },
        },
      });
      return NextResponse.json({ message: "Dependency removed" });
    } catch {
      return NextResponse.json(
        { error: "Dependency not found" },
        { status: 404 },
      );
    }
  }
}

```

- ☐ **Step 3: Create the graph data API route**

Create `src/app/api/todos/graph/route.ts`:

```

import { NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";
import { findCriticalPath, computeEarliestStartDates } from "@/lib/graph";

export async function GET() {
  const todos = await prisma.todo.findMany({
    include: { dependsOn: true },
  });

  const criticalPath = findCriticalPath(todos);
  const earliestStartDates = computeEarliestStartDates(todos);

  return NextResponse.json({ criticalPath, earliestStartDates });
}

```


- ☐ **Step 4: Update GET /api/todos to include dependency data**

In `src/app/api/todos/route.ts`, update the GET handler:

```
export async function GET() {
  try {
    const todos = await prisma.todo.findMany({
      orderBy: { createdAt: "desc" },
      include: {
        dependsOn: {
          include: { dependency: { select: { id: true,
        },
        dependedOnBy: {
          include: { dependent: { select: { id: true,
        },
      },
    });
    return NextResponse.json(todos);
  } catch {
    return NextResponse.json({ error: "Error fetching
  }
}
```

- ☐ **Step 5: Update DELETE /api/todos/[id] to cascade-delete dependencies**

Replace the entire contents of `src/app/api/todos/[id]/route.ts` (updating to async params and adding dependency cascade):

```
import { NextResponse } from "next/server";
import { prisma } from "@/lib/prisma";

export async function DELETE(
  _request: Request,
  { params }: { params: Promise<{ id: string }> },
) {
  const { id: rawId } = await params;
  const id = parseInt(rawId);
  if (isNaN(id)) {
```

```

    return NextResponse.json({ error: "Invalid ID" },
  }

  try {
    await prisma.todoDependency.deleteMany({
      where: {
        OR: [{ dependentId: id }, { dependencyId: id
      },
    });
    await prisma.todo.delete({ where: { id } });
    return NextResponse.json({ message: "Todo deleted"
  } catch {
    return NextResponse.json({ error: "Error deleting
  }
}

```

- ☐ **Step 6: Verify dependency APIs with curl**

Create two todos, add a dependency, verify circular detection:

```

# Create todos
curl -s -X POST http://localhost:3000/api/todos -H 'C
curl -s -X POST http://localhost:3000/api/todos -H 'C

# Add dependency: B depends on A (use actual IDs from
curl -s -X POST http://localhost:3000/api/todos/2/depe

# Try circular: A depends on B (should fail)
curl -s -X POST http://localhost:3000/api/todos/1/depe

# Verify graph data
curl -s http://localhost:3000/api/todos/graph | jq .

```

Expected: Circular dependency returns 400 with error message.

Graph returns criticalPath and earliestStartDates.

- ☐ **Step 7: Commit**

```
git add src/lib/graph.ts src/app/api/todos/  
git commit -m "feat: implement task dependency backend"
```

Task 5: Dependencies Frontend + Todo Cards

Files:

- Create: `src/components/ToDoCard.tsx`
- Modify: `src/app/page.tsx`
- ☐ **Step 1: Install React Flow and dagre**

```
npm install @xyflow/react dagre  
npm install -D @types/dagre
```

- ☐ **Step 2: Create the ToDoCard component**

Create `src/components/ToDoCard.tsx`:

```
"use client";  
  
import { useState } from "react";  
import Image from "next/image";  
  
interface DependencyInfo {  
  dependencyId: number;  
  dependency: { id: number; title: string };  
}  
  
interface Todo {  
  id: number;  
  title: string;  
  createdAt: string;
```

```

    dueDate: string | null;
    imageUrl: string | null;
    dependsOn: DependencyInfo[];
    dependedOnBy: { dependentId: number; dependent: { id: number; }
  }

interface TodoCardProps {
  todo: Todo;
  allTodos: Todo[];
  onDelete: () => void;
  onAddDependency: (depId: number) => void;
  onRemoveDependency: (depId: number) => void;
  earliestStartDate?: string;
  isOnCriticalPath?: boolean;
}

export default function TodoCard({
  todo,
  allTodos,
  onDelete,
  onAddDependency,
  onRemoveDependency,
  earliestStartDate,
  isOnCriticalPath,
}: TodoCardProps) {
  const [imageLoaded, setImageLoaded] = useState(false);

  const isOverdue = todo.dueDate && new Date(todo.dueDate).getTime() < Date.now();
  const availableDeps = allTodos.filter(
    (t) =>
      t.id !== todo.id &&
      !todo.dependsOn.some((d) => d.dependencyId === t.id)
  );

  return (
    <div
      className={`overflow-hidden rounded-xl bg-white
        isOnCriticalPath ? "ring-2 ring-orange-400" :
      }`
    >
      </* Image */>
      {todo.imageUrl && (
        <div className="relative h-40 w-full bg-gray-100" >
          <div className="absolute inset-0 animate-pulse" >

```

```

    })
    <Image
      src={todo.imageUrl}
      alt={todo.title}
      width={600}
      height={400}
      className={`h-40 w-full object-cover transition-opacity
        imageLoaded ? "opacity-100" : "opacity-0"}
      `}
      onLoad={() => setImageLoaded(true)}
    />
  </div>
)}

<div className="p-4">
  <div className="flex items-start justify-between">
    <div className="min-w-0 flex-1">
      {/* Title */}
      <div className="flex items-center gap-2">
        <h3 className="truncate text-lg font-semibold">
          {todo.title}
        </h3>
        {isOnCriticalPath && (
          <span className="shrink-0 rounded-full text-center text-sm">
            Critical Path
          </span>
        )}
      </div>

      {/* Due Date */}
      {todo.dueDate && (
        <p
          className={`text-sm ${
            isOverdue ? "font-semibold text-red-500" : ""
          }`}
        >
          Due: {new Date(todo.dueDate).toLocaleDateString(
            {isOverdue && " (overdue)"}
          )}
        </p>
      )}

      {/* Earliest Start Date */}
      {earliestStartDate && (
        <p className="text-xs text-blue-500">
          Earliest start:{" "}
        </p>
      )}
    </div>
  </div>

```

```

        {new Date(earliestStartDate).toLocaleDateString()}
      </p>
    )}

    {/* Dependencies */}
    {todo.dependsOn.length > 0 && (
      <div className="mt-2 flex flex-wrap items-center">
        <span className="text-xs text-gray-400">Depends on</span>
        {todo.dependsOn.map((dep) => (
          <span
            key={dep.dependencyId}
            className="inline-flex items-center gap-1"
          >
            {dep.dependency.title}
            <button
              onClick={() => onRemoveDependency(dep)}
              className="ml-0.5 hover:text-red-500"
            >
              ✕
            </button>
          </span>
        ))}
      </div>
    )}

    {/* Depended on by */}
    {todo.dependedOnBy.length > 0 && (
      <div className="mt-1 flex flex-wrap items-center">
        <span className="text-xs text-gray-400">Depended on by</span>
        {todo.dependedOnBy.map((dep) => (
          <span
            key={dep.dependentId}
            className="rounded-full bg-gray-100 px-2 py-1"
          >
            {dep.dependent.title}
          </span>
        ))}
      </div>
    )}

    {/* Add Dependency */}
    {availableDeps.length > 0 && (
      <select
        onChange={(e) => {
          if (e.target.value) {
            onAddDependency(e.target.value);
          }
        }}
      >
        {availableDeps.map((dep) => (
          <option key={dep.id} value={dep.id}>{dep.title}</option>
        ))}
      </select>
    )}
  )}

```

```

        onAddDependency(parseInt(e.target
        e.target.value = "";
    }
    })
    className="mt-2 rounded border border-
    defaultValue=""
  >
    <option value="" disabled>
      + Add dependency...
    </option>
    {availableDeps.map((t) => (
      <option key={t.id} value={t.id}>
        {t.title}
      </option>
    ))}
  </select>
  })
</div>

{/* Delete Button */}
<button
  onClick={onDelete}
  className="ml-4 shrink-0 text-red-400 tra
  >
    <svg
      className="h-6 w-6"
      fill="none"
      stroke="currentColor"
      viewBox="0 0 24 24"
    >
      <path
        strokeLinecap="round"
        strokeLinejoin="round"
        strokeWidth={2}
        d="M6 18L18 6M6 6l12 12"
      />
    </svg>
  </button>
</div>
</div>
</div>
);
}

```

- ☐ **Step 3: Rewrite page.tsx as the orchestrator**

Replace `src/app/page.tsx` entirely:

```
"use client";

import { useState, useEffect, useCallback } from "react";
import TodoCard from "@components/TodoCard";

interface DependencyInfo {
  dependencyId: number;
  dependency: { id: number; title: string };
}

interface Todo {
  id: number;
  title: string;
  createdAt: string;
  dueDate: string | null;
  imageUrl: string | null;
  dependsOn: DependencyInfo[];
  dependedOnBy: {
    dependentId: number;
    dependent: { id: number; title: string };
  }[];
}

interface GraphData {
  criticalPath: number[];
  earliestStartDates: Record<number, string>;
}

export default function Home() {
  const [todos, setTodos] = useState<Todo[]>([]);
  const [newTodo, setNewTodo] = useState("");
  const [dueDate, setDueDate] = useState("");
  const [isCreating, setIsCreating] = useState(false);
  const [view, setView] = useState<"list" | "graph">("list");
  const [graphData, setGraphData] = useState<GraphData>({
    criticalPath: [],
    earliestStartDates: {},
  });

  const fetchData = useCallback(async () => {
    try {
      const [todosRes, graphRes] = await Promise.all([
        fetch("http://localhost:3001/todos").then((res) => res.json()),
        fetch("http://localhost:3001/graph").then((res) => res.json()),
      ]);
      setTodos(todosRes);
      setGraphData(graphRes);
    } catch (error) {
      console.error("Error fetching data:", error);
    }
  }, [setTodos, setGraphData]);

  useEffect(() => {
    fetchData();
  }, [fetchData]);

  const handleCreate = () => {
    if (!newTodo || !dueDate) return;
    const newTodoObj: Todo = {
      id: Date.now(),
      title: newTodo,
      createdAt: new Date().toISOString(),
      dueDate: dueDate,
      imageUrl: null,
      dependsOn: [],
      dependedOnBy: [],
    };
    setTodos([...todos, newTodoObj]);
    setNewTodo("");
    setDueDate("");
    setIsCreating(false);
  };

  const handleToggleView = () => {
    setView(view === "list" ? "graph" : "list");
  };
}
```



```

        fetch("/api/todos"),
        fetch("/api/todos/graph"),
    ]);
    if (todosRes.ok) setTodos(await todosRes.json());
    if (graphRes.ok) setGraphData(await graphRes.json());
  } catch (error) {
    console.error("Failed to fetch data:", error);
  }
}, []);

useEffect(() => {
  fetchData();
}, [fetchData]);

const handleAddTodo = async () => {
  if (!newTodo.trim()) return;
  setIsCreating(true);
  try {
    const res = await fetch("/api/todos", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title: newTodo, dueDate: newDueDate }),
    });
    if (res.ok) {
      setNewTodo("");
      setDueDate("");
      fetchData();
    }
  } finally {
    setIsCreating(false);
  }
};

const handleDeleteTodo = async (id: number) => {
  await fetch(`/api/todos/${id}`, { method: "DELETE" });
  fetchData();
};

const handleAddDependency = async (
  todoId: number,
  dependencyId: number,
) => {
  const res = await fetch(`/api/todos/${todoId}/dependencies`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ dependencyId }),
  });
  if (res.ok) {
    fetchData();
  }
};

```

```

        body: JSON.stringify({ dependencyId }),
    });
    if (res.ok) {
        fetchData();
    } else {
        const data = await res.json();
        alert(data.error);
    }
};

const handleRemoveDependency = async (
    todoId: number,
    dependencyId: number,
) => {
    await fetch(`/api/todos/${todoId}/dependencies`,
        method: "DELETE",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ dependencyId }),
    );
    fetchData();
};

const criticalPathSet = new Set(graphData?.criticall

return (
    <div className="flex min-h-screen flex-col items-c
    <div className="w-full max-w-2xl">
        <h1 className="mb-8 text-center text-4xl font-
            Things To Do App
        </h1>

        { /* Create Form */ }
        <div className="mb-6 flex overflow-hidden rou
            <input
                type="text"
                className="min-w-0 flex-1 p-3 pl-5 text-g
                placeholder="Add a new todo..."
                value={newTodo}
                onChange={(e) => setNewTodo(e.target.valu
                onKeyDown={(e) => e.key === "Enter" && ha
            />
            <input
                type="date"
                value={dueDate}
                onChange={(e) => setDueDate(e.target.valu

```

```

        className="border-x border-gray-200 px-3 +
    />
    <button
      onClick={handleAddTodo}
      disabled={isCreating || !newTodo.trim()}
      className="px-6 text-indigo-600 transition
    >
      {isCreating ? "... " : "Add"}
    </button>
  </div>

  {/* View Toggle */}
  <div className="mb-4 flex justify-end">
    <button
      onClick={() => setView(view === "list" ?
      className="rounded-lg bg-white/20 px-4 py-
    >
      {view === "list"
        ? "View Dependency Graph"
        : "Back to List"}
    </button>
  </div>

  {/* Content */}
  {view === "list" ? (
    <div className="space-y-4">
      {todos.map((todo) => (
        <TodoCard
          key={todo.id}
          todo={todo}
          allTodos={todos}
          onDelete={() => handleDeleteTodo(todo)
          onAddDependency={ (depId) =>
            handleAddDependency(todo.id, depId)
          }
          onRemoveDependency={ (depId) =>
            handleRemoveDependency(todo.id, dep
          }
          earliestStartDate={
            graphData?.earliestStartDates[todo..
          }
          isOnCriticalPath={criticalPathSet.has
        />
      )}}
      {todos.length === 0 && (

```

```

        <p className="text-center text-white/80" >
          No todos yet. Add one above!
        </p>
      )}
    </div>
  ) : (
    <div
      className="overflow-hidden rounded-xl bg-white"
      style={{ height: "600px" }}
    >
      {/* DependencyGraph will be added in Task 5
      <div className="flex h-full items-center justify-center" >
        Graph view — coming in Task 6
      </div>
    </div>
  )}
</div>
</div>
);
}

```

- ☐ **Step 4: Verify in browser**

Start dev server, test:

1. Create todos with and without due dates — images load with fade-in
2. Add dependencies via dropdown — chips appear, circular deps show alert
3. Remove dependencies via × button
4. Delete a todo that has dependencies — no errors
5. "Blocks" labels appear on prerequisite tasks

- ☐ **Step 5: Commit**

```

git add src/components/ToDoCard.tsx src/app/page.tsx
git commit -m "feat: add todo cards with images, dependencies"

```

Task 6: Dependency Graph Visualization

Files:

- Create: `src/components/DependencyGraph.tsx`
- Modify: `src/app/page.tsx`
- ☐ **Step 1: Create the DependencyGraph component**

Create `src/components/DependencyGraph.tsx` :

```
"use client";

import { useMemo } from "react";
import {
  ReactFlow,
  Background,
  Controls,
  Handle,
  Position,
  MarkerType,
} from "@xyflow/react";
import dagre from "dagre";
import "@xyflow/react/dist/style.css";

interface Todo {
  id: number;
  title: string;
  dueDate: string | null;
  dependsOn: { dependencyId: number }[];
}

interface DependencyGraphProps {
  todos: Todo[];
  criticalPath: number[];
  earliestStartDates: Record<number, string>;
}

const NODE_WIDTH = 220;
const NODE_HEIGHT = 100;

function TodoNode({
```

```

    data,
  }: {
    data: {
      todo: Todo;
      isOnCriticalPath: boolean;
      earliestStartDate?: string;
    };
  }) {
    const { todo, isOnCriticalPath, earliestStartDate }
    const isOverdue = todo.dueDate && new Date(todo.dueDate).getTime() < Date.now()

    return (
      <div
        className={`rounded-lg border-2 bg-white p-3 shadow-md ${
          isOnCriticalPath
            ? "border-orange-500 bg-orange-50"
            : "border-gray-200"
        }`}
        style={{ width: NODE_WIDTH }}
      >
        <Handle
          type="target"
          position={Position.Top}
          className="!bg-gray-400"
        />
        <p className="truncate text-sm font-semibold text-gray-700">
          {todo.title}
        </p>
        {todo.dueDate && (
          <p
            className={`text-xs ${
              isOverdue ? "font-bold text-red-500" : "text-gray-500"
            }`}
          >
            Due: {new Date(todo.dueDate).toLocaleDateString()}
          </p>
        )}
        {earliestStartDate && (
          <p className="text-xs text-blue-500">
            Start: {new Date(earliestStartDate).toLocaleDateString()}
          </p>
        )}
        {isOnCriticalPath && (
          <p className="mt-1 text-xs font-medium text-orange-500">
            Critical Path
          </p>
        )}
      </div>
    )
  }
}

```

```

        </p>
    )}
    <Handle
        type="source"
        position={Position.Bottom}
        className="!bg-gray-400"
    />
</div>
);
}

const nodeTypes = { todoNode: TodoNode };

export default function DependencyGraph({
    todos,
    criticalPath,
    earliestStartDates,
}: DependencyGraphProps) {
    const { nodes, edges } = useMemo(() => {
        if (todos.length === 0) return { nodes: [], edges

// Build critical path lookup
const cpSet = new Set(criticalPath);
const cpEdges = new Set<string>();
for (let i = 0; i < criticalPath.length - 1; i++)
    cpEdges.add(`${criticalPath[i]}-${criticalPath[i+1]}`);

// Dagre layout
const g = new dagre.graphlib.Graph();
g.setGraph({ rankdir: "TB", nodesep: 60, ranksep:
g.setDefaultEdgeLabel(() => ({})));

for (const todo of todos) {
    g.setNode(String(todo.id), {
        width: NODE_WIDTH,
        height: NODE_HEIGHT,
    });
}

const flowEdges: any[] = [];
for (const todo of todos) {
    for (const dep of todo.dependsOn) {
        g.setEdge(String(dep.dependencyId), String(todo.id), {
            const edgeKey = `${dep.dependencyId}-${todo.id}`

```

```

        const isOnCP = cpEdges.has(edgeKey);
        flowEdges.push({
            id: `e${edgeKey}`,
            source: String(dep.dependencyId),
            target: String(todo.id),
            animated: isOnCP,
            style: {
                stroke: isOnCP ? "#f97316" : "#94a3b8",
                strokeWidth: isOnCP ? 2.5 : 1.5,
            },
            markerEnd: {
                type: MarkerType.ArrowClosed,
                color: isOnCP ? "#f97316" : "#94a3b8",
            },
        });
    }
}

dagre.layout(g);

const flowNodes = todos.map((todo) => {
    const pos = g.node(String(todo.id));
    return {
        id: String(todo.id),
        type: "todoNode" as const,
        position: {
            x: pos.x - NODE_WIDTH / 2,
            y: pos.y - NODE_HEIGHT / 2,
        },
        data: {
            todo,
            isOnCriticalPath: cpSet.has(todo.id),
            earliestStartDate: earliestStartDates[todo.id],
        },
    };
});

return { nodes: flowNodes, edges: flowEdges };
}, [todos, criticalPath, earliestStartDates]);

if (todos.length === 0) {
    return (
        <div className="flex h-full items-center justify-center">
            No todos yet. Add some to see the graph.
        </div>
    );
}

```



```

    );
  }

  const hasDeps = todos.some((t) => t.dependsOn.length);
  if (!hasDeps) {
    return (
      <div className="flex h-full items-center justify-center">
        No dependencies defined. Add dependencies to the graph.
      </div>
    );
  }

  return (
    <ReactFlow
      nodes={nodes}
      edges={edges}
      nodeTypes={nodeTypes}
      fitView
      nodesDraggable={false}
      nodesConnectable={false}
      proOptions={{ hideAttribution: true }}
    >
      <Background />
      <Controls showInteractive={false} />
    </ReactFlow>
  );
}

```

- ❑ **Step 2: Wire DependencyGraph into page.tsx**

In `src/app/page.tsx`, add the import:

```
import DependencyGraph from "@components/DependencyG";
```

Replace the placeholder graph `<div>` (the one saying "Graph view — coming in Task 6") with:

```

<div
  className="overflow-hidden rounded-xl bg-white shadow
  style={{ height: "600px" }}
>
  <DependencyGraph
    todos={todos}
    criticalPath={graphData?.criticalPath ?? []}
    earliestStartDates={graphData?.earliestStartDates}
  />
</div>

```

- ☐ **Step 3: Verify graph visualization**

In the browser:

1. Create 3-4 todos with due dates
2. Add dependencies to form a chain ($A \rightarrow B \rightarrow C$)
3. Click "View Dependency Graph"
4. Confirm: nodes laid out top-to-bottom, edges with arrows, critical path highlighted in orange with animated edges
5. Earliest start dates shown on nodes
6. Controls (zoom in/out/fit) work
7. Toggle back to list view works

- ☐ **Step 4: Commit**

```

git add src/components/DependencyGraph.tsx src/app/page.tsx
git commit -m "feat: add dependency graph visualization"

```

Task 7: Polish & README

Files:

- Modify: `README.md`

- ☐ **Step 1: Run linting and fix any issues**

```
npm run lint:fix  
npm run format:fix
```

Fix any remaining issues.

- ☐ **Step 2: Full end-to-end verification**

Test the complete flow:

1. Create todo "Buy groceries" with due date tomorrow → image loads, date shown in gray
2. Create todo "Meal prep" with due date yesterday → date shown in red, "(overdue)"
3. Create todo "Cook dinner" with no date → no date shown
4. Add dependency: "Cook dinner" depends on "Buy groceries" → chip appears
5. Add dependency: "Meal prep" depends on "Cook dinner" → chain formed
6. Try circular: "Buy groceries" depends on "Meal prep" → alert with error
7. Switch to graph view → three nodes in chain, critical path highlighted
8. Earliest start dates correct on all nodes and cards
9. Delete "Cook dinner" → dependencies cleaned up, graph updates
10. Toggle between list/graph views works smoothly

- ☐ **Step 3: Add Solution section to README**

Add the following to the end of `README.md` (above the submission section):

Solution

Part 1: Due Dates

Added optional due date field to the todo creation form.

Part 2: Image Previews

When a todo is created, the server searches the Pexels API for a matching image.

Part 3: Task Dependencies

Implemented a full dependency system:

- **Multiple dependencies:** Each task can depend on multiple other tasks.
- **Circular dependency prevention:** Server-side cycle detection.
- **Critical path:** Computed server-side using topological sorting.
- **Earliest start dates:** Calculated as the latest completion date of all dependencies.
- **Dependency graph:** Interactive visualization using D3.js.

Screenshot

<!-- Add screenshot here -->

- ☐ **Step 4: Take a screenshot and add it**

Take a screenshot of the app with todos, dependencies, and the graph view. Save it to the repo and reference it in the README.

- ☐ **Step 5: Final commit**

```
git add -A
git commit -m "docs: add solution section to README"
```